# Practical 1 – Shell Programming

```
#!/bin/bash
FILE="addressbook.txt"
while true
do
    echo "----- ADDRESS BOOK -----"
    echo "1. Create"
    echo "2. View"
    echo "3. Insert"
    echo "4. Delete"
    echo "5. Modify"
    echo "6. Exit"
    echo "Enter your choice:"
    read ch

    case $ch in

    1) > $FILE
        echo "Address book created."
        ;;

    2) echo "Address Book:"
        cat $FILE
        ;;

    3) echo "Enter ID:"
        read id
        echo "Enter Name:"
        read name
        echo "Enter Phone:"
        read phone

        echo "$id $name $phone" >> $FILE
        echo "Record inserted."
        ;;

    4) echo "Enter ID to delete:"
        read id
        grep -v "^$id " $FILE > temp
        mv temp $FILE
        echo "Record deleted."
        ;;

    5) echo "Enter ID to modify:"
        read id
        grep -v "^$id " $FILE > temp

        echo "Enter New Name:"
        read name
        echo "Enter New Phone:"
        read phone

        echo "$id $name $phone" >> temp
        mv temp $FILE

        echo "Record modified."
        ;;

    6) exit
        ;;
    *)
        echo "Invalid choice."
        ;;

    esac
done
```

# Practical 2 – Process Management

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid < 0)
    {
        printf("Process creation failed\n");
    }
    else if(pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

# Practical 3 – Producer-Consumer Problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    int item;

    for(item = 1; item <= 5; item++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg)
{
    int item;

    for(int i = 1; i <= 5; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        sleep(1);
    }

    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    return 0;
}
```

# Practical 4 – Dining Philosophers Problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

pthread_mutex_t fork_mutex[N];
sem_t room;

void *philosopher(void *arg)
{
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % N;

    printf("Philosopher %d is thinking\n", id);

    sleep(1);

    sem_wait(&room);

    pthread_mutex_lock(&fork_mutex[left]);
    pthread_mutex_lock(&fork_mutex[right]);

    printf("Philosopher %d is eating\n", id);

    sleep(2);

    pthread_mutex_unlock(&fork_mutex[right]);
    pthread_mutex_unlock(&fork_mutex[left]);

    sem_post(&room);

    printf("Philosopher %d finished eating\n", id);

    return NULL;
}

int main()
{
    pthread_t philosophers[N];
    int id[N];

    sem_init(&room, 0, N - 1);

    for(int i = 0; i < N; i++)
        pthread_mutex_init(&fork_mutex[i], NULL);

    for(int i = 0; i < N; i++)
    {
        id[i] = i;
        pthread_create(&philosophers[i], NULL,
                       philosopher, &id[i]);
    }

    for(int i = 0; i < N; i++)
        pthread_join(philosophers[i], NULL);

    return 0;
}
```

# Practical 5 – Banker’s Algorithm

```
#include <stdio.h>

int main()
{
    int n, m;
    int alloc[10][10], max[10][10], need[10][10];
    int avail[10], finish[10] = {0};
    int safe[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    printf("Enter Allocation Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");
    for(int j = 0; j < m; j++)
        scanf("%d", &avail[j]);

    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    int count = 0;

    while(count < n)
    {
        int found = 0;

        for(int i = 0; i < n; i++)
        {
            if(finish[i] == 0)
            {
                int possible = 1;

                for(int j = 0; j < m; j++)
                {
                    if(need[i][j] > avail[j])
                    {
                        possible = 0;
                        break;
                    }
                }

                if(possible)
                {
                    for(int j = 0; j < m; j++)
                        avail[j] += alloc[i][j];

                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(!found)
            break;
    }

    if(count == n)
    {
        printf("System is in SAFE state.\n");
        printf("Safe Sequence: ");

        for(int i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    }
    else
    {
        printf("System is NOT in safe state.\n");
    }

    return 0;
}
```

# Practical 6 – IPC Using FIFOs

```
/* Process 1 */

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd1, fd2;
    char str[500];
    char result[200];

    mkfifo("fifo1", 0666);
    mkfifo("fifo2", 0666);

    fd1 = open("fifo1", O_WRONLY);

    printf("Enter a sentence: ");
    fgets(str, sizeof(str), stdin);

    write(fd1, str, strlen(str) + 1);
    close(fd1);

    fd2 = open("fifo2", O_RDONLY);

    read(fd2, result, sizeof(result));
    printf("\nResult from Process 2:\n%s\n", result);

    close(fd2);

    return 0;
}


/* Process 2 */

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <ctype.h>
#include <string.h>

int main()
{
    int fd;
    char str[500];
    int characters = 0;
    int words = 0;
    int lines = 0;
    int inword = 0;
    char result[200];

    fd = open("fifo1", O_RDONLY);

    read(fd, str, sizeof(str));
    close(fd);

    for(int i = 0; str[i] != '\0'; i++)
    {
        characters++;

        if(str[i] == '\n')
            lines++;

        if(isspace(str[i]))
            inword = 0;
        else if(inword == 0)
        {
            words++;
            inword = 1;
        }
    }

    FILE *fp = fopen("output.txt", "w");
    fprintf(fp, "%s", str);
    fclose(fp);

    sprintf(result,
            "Characters = %d\nWords = %d\nLines = %d",
            characters, words, lines);

    fd = open("fifo2", O_WRONLY);
    write(fd, result, strlen(result) + 1);
    close(fd);

    return 0;
}
```

# Practical 7 – Full Duplex Communication Using Pipes

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <fcntl.h>

int main()
{
    int pipe1[2], pipe2[2];
    pid_t pid;
    char filename[100];
    char buffer[1000];

    pipe(pipe1);
    pipe(pipe2);

    pid = fork();

    if(pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if(pid > 0)
    {
        close(pipe1[0]);
        close(pipe2[1]);

        printf("Enter file name: ");
        scanf("%s", filename);

        write(pipe1[1], filename, strlen(filename) + 1);
        close(pipe1[1]);

        int n = read(pipe2[0], buffer, sizeof(buffer) - 1);
        buffer[n] = '\0';

        printf("\nFile Contents:\n%s\n", buffer);

        close(pipe2[0]);

        wait(NULL);
    }
    else
    {
        close(pipe1[1]);
        close(pipe2[0]);

        read(pipe1[0], filename, sizeof(filename));
        close(pipe1[0]);

        int fd = open(filename, O_RDONLY);

        if(fd < 0)
        {
            char msg[] = "File cannot be opened.";
            write(pipe2[1], msg, strlen(msg) + 1);
        }
        else
        {
            int n = read(fd, buffer, sizeof(buffer) - 1);
            buffer[n] = '\0';

            write(pipe2[1], buffer, strlen(buffer) + 1);

            close(fd);
        }

        close(pipe2[1]);
    }

    return 0;
}
```

# Practical 8 – Disk Scheduling Algorithms

```
#include <stdio.h>
#include <stdlib.h>

void sort(int a[], int n)
{
    int i, j, temp;

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

void sstf(int a[], int n, int head)
{
    int visited[100] = {0};
    int total = 0;

    printf("\nSSTF Sequence: ");

    for(int i = 0; i < n; i++)
    {
        int min = 9999;
        int index = -1;

        for(int j = 0; j < n; j++)
        {
            if(!visited[j])
            {
                int distance = abs(head - a[j]);

                if(distance < min)
                {
                    min = distance;
                    index = j;
                }
            }
        }

        visited[index] = 1;
        total += abs(head - a[index]);
        head = a[index];

        printf("%d ", head);
    }

    printf("\nTotal Head Movement = %d\n", total);
}

void scan(int a[], int n, int head, int disk_size)
{
    int total = 0;
    int b[100];

    for(int i = 0; i < n; i++)
        b[i] = a[i];

    sort(b, n);

    printf("\nSCAN Sequence: ");

    int pos = 0;

    while(pos < n && b[pos] < head)
        pos++;

    for(int i = pos; i < n; i++)
    {
        printf("%d ", b[i]);
        total += abs(head - b[i]);
        head = b[i];
    }

    total += abs(head - (disk_size - 1));
    head = disk_size - 1;

    for(int i = pos - 1; i >= 0; i--)
    {
        printf("%d ", b[i]);
        total += abs(head - b[i]);
        head = b[i];
    }

    printf("\nTotal Head Movement = %d\n", total);
}

void clook(int a[], int n, int head)
{
    int b[100];
    int total = 0;

    for(int i = 0; i < n; i++)
        b[i] = a[i];

    sort(b, n);
    printf("\nC-LOOK Sequence: ");
```

```
    int pos = 0;

    while(pos < n && b[pos] < head)
        pos++;

    for(int i = pos; i < n; i++)
    {
        printf("%d ", b[i]);
        total += abs(head - b[i]);
        head = b[i];
    }

    if(pos > 0)
    {
        total += abs(head - b[0]);
        head = b[0];

        for(int i = 0; i < pos; i++)
        {
            printf("%d ", b[i]);

            if(i > 0)
                total += abs(head - b[i]);

            head = b[i];
        }
    }

    printf("\nTotal Head Movement = %d\n", total);
}

int main()
{
    int n, head, disk_size;
    int a[100];

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter disk size: ");
    scanf("%d", &disk_size);

    sstf(a, n, head);
    scan(a, n, head, disk_size);
    clook(a, n, head);

    return 0;
}
```